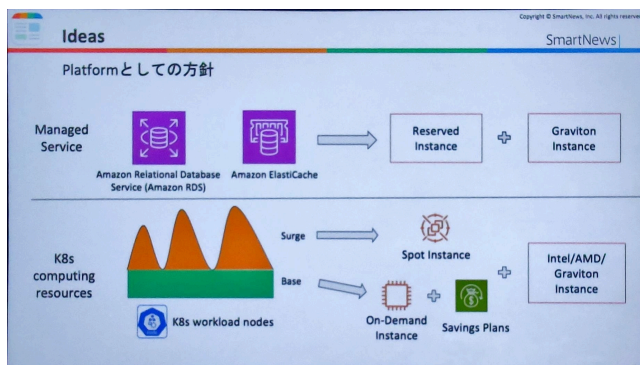
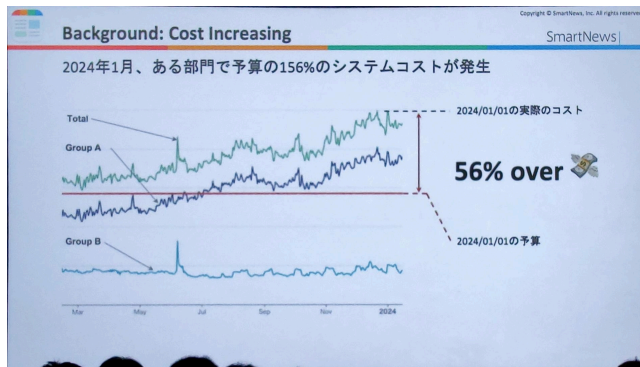


SmartNews k8s大規模導入

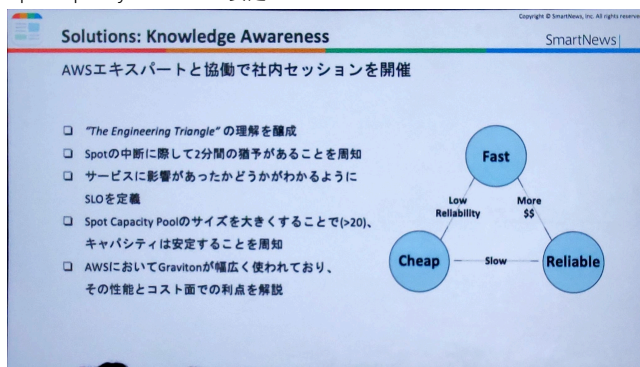
2025年6月26日 15:10

コスト増加が問題になっていた

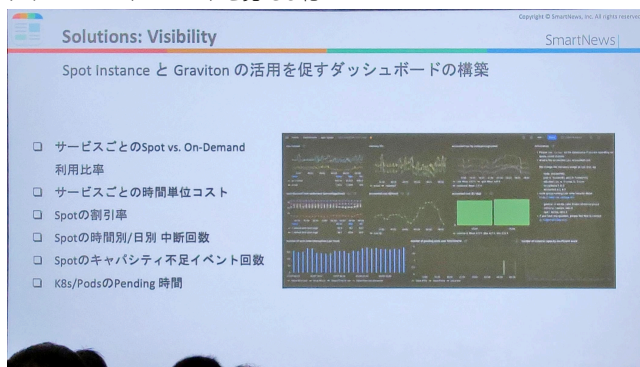


Spotインスタンスは中断2分前には少なくとも通知が来る
→どこにくる？

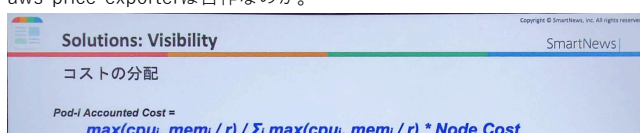
SpotCapacityPool >20で安定

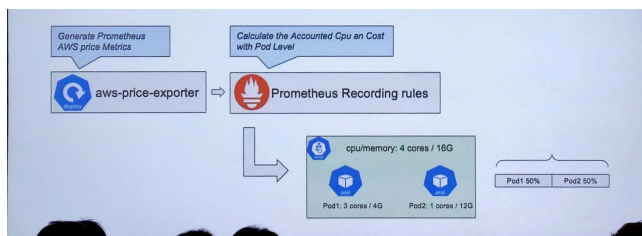


ダッシュボードでコストを見える化



aws-price-exporterは自作なのか。





Spotは割引率が最大でなくてもいいので、中断率が低いものを選ぶ



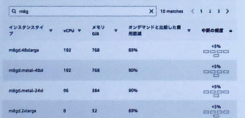

Copyright © SmartNews, All Rights Reserved.

SmartNews |

Solutions: Use selected instance types

Spotの入手可能性とコストのバランス

- ❑ AWSのSpot Instance Advisorから利用可能なInstance Typeを確認
- ❑ 中断の頻度とコストを見ながら適切なタイプを選択
 - 中断の頻度 < 15%
 - オンデマンドと比較した費用削減 > 50%



インスタンスタイプ	コスト/月	中断率	中断率の目安
m5.xlarge	192 768	91%	<15%
m5g.xlarge	192 768	91%	<15%
m5.xlarge	192 768	91%	<15%
m5g.xlarge	192 768	91%	<15%
m5.xlarge	192 768	91%	<15%
m5g.xlarge	192 768	91%	<15%



その結果Capacity Insufficient イベントは大きく減少

Solutions: Using Karpetner to improve scaling performance

SmartNews |

Cluster AutoscalerからKarpetnerへの移行

Cluster Autoscaler:

CAIS(AWS Auto Scaling Group(ASG))に依存して新しいインスタンスをプロビジョニングするため、AWSがASGの制限に数分かかる。
この遅延の主な理由は、AWSが新しいインスタンスをプロビジョニングし、それらのインスタンスがクラス内で準備完了になるまでに時間がかかるため。

Karpetner:

Karpetnerは多くの場合、1分以内にインスタンスをプロビジョニングできるほど、はるかに迅速にスケールアップするように設計されている。
これは、KarpetnerがEC2 APIと直接やり取りし、より低いVSGメモリランをバイパスするため。

スポットインスタンスを選択する上で、スケールアップのパフォーマンスは非常に重要。なぜなら、前述の通知は3分しかかず、Fallbackの時間を考慮する必要があるため。

インスタンスを上げるために
Auto Scaling Group (ASG)に依存

EC2 APIと直接やりとり

<https://www.mpsia.co.jp/blog/karpetner-to-improve-cluster-autoscaler-vs-ca/>

Karpenterのカスタマイズ

→Karpenterのバージョンアップ時はどうしてるんだろう。




Solutions: Use Karmpenter to improve scaling performance

Copyright © SmartNews, Inc. All rights reserved. SmartNews |

OSSであるKarmpenterを独自でカスタマイズし、効率と安定性を更に向上

- スケールダウンのトリガーとして“利用率”を指定
- 直近3分以内に配置されたPending状態のPodsはスケールアップ計画の計算から除外
- Spot Instance中断が発生した際には、即座にNodeClaimsを作成

サービス全体の安定性を向上

NodeClaimsの繰り返し作成を回避

Spot中断に備え時間短縮

スケールアップにかかる時間の短縮に成功

6 Minutes

3 Minutes

pricing: spot を指定してSpot優先（スポットが起動しない場合はオンデマンドを利用数r）

Copyright © SmartNews, Inc. All rights reserved.

SmartNews |

Solutions: Spot Fallback

最大限Spotを活用するよう適切にコントロールされたFallback

Karpenterの機能を利用:

spec.expireAfter: 1h; spec.terminationGracePeriod: 5m

```

    graph LR
      NS[Node Selector] --> P1["pricing: spot  
architecture: arm  
Generation: 8"]
      P1 --> MN[Matching Node]
      MN --> NP["pricing: spot  
architecture: arm  
Generation: 8  
Weight: 80  
expireAfter: 1h  
capacityType: ON_DEMAND"]
      NP --> LN[Launch Node]
      LN -- Success --> NP
      subgraph Karpenter
        NP
        LN
      end
      Karpenter -- "Check capacity again with a fixed interval" --> NP
  
```

- 明示的にSpot Instanceを指定できる
- Fallback先のOn-Demandインスタンスは一定期間後で停止

**Ensure Capacity
Keep Cost Efficiency**

Spot適用基準の作成

Solutions: Seamless migration

システム重要度	選択基準	Spot Instanceへの適性
Critical System	トランザクション実行時間 < 30秒 (e.g. オンラインサービスの応答時間 < 1秒)	✓ 適合 – 実行時間が2分より短ければ、Spot中断による処理の強制停止を回避できる
	トランザクション実行時間 > 2分 (e.g. 時間のかかるオフラインジョブ)	✗ 不適合 – 処理中に強制停止される
	Pod起動時間 < 1分	✓ 適合 – 再起動しても即座にリカバリできる
	Pod起動時間 > 1分 & 10+レプリカ	✓ 許容 – 起動に時間がかかるとしても、レプリカが相当数あれば強制停止の影響を緩和可能
Non-Critical System	(特になし)	✓ 適合 – 早速Spot Instanceを導入

Solutions: Seamless migration Copyright © SmartNews, Inc. All rights reserved. SmartNews |

Gravitonもサポートするためにマルチプラットフォームイメージを活用

- 優れたパフォーマンス

167.2ms	115.8ms
248.7ms	169.6ms
- 低いコスト

m6a.Bxlarge: \$0.3955 hourly
m6i.Bxlarge: \$0.5882 hourly
m6a.Bxlarge: \$0.6439 hourly
- Intel, AMD, Gravitonのより柔軟なキャパシティを実現

The charts show performance metrics (latency) and average spot prices (\$/hour) for different instance types (general, memory, intel, amd) across various AWS regions. The performance chart shows that Graviton instances (m6a) generally have lower latency than Intel-based instances (m6i). The cost chart shows that Spot prices fluctuate over time, with m6a instances generally being cheaper than m6i instances.

